What is difference between Titan and Neo4j graph database?

6

1

I had worked on relational database; but now want to learn about graph database. I came to know that these two are graph database. What is difference between these two databases. What should we prefer among them?

database neo4j graph-databases titan non-relational-database

edited Jul 29 '18 at 18:16
Cœur
21.5k 10 125 171

asked Jul 27 '17 at 10:06
Akshay Gaikwad
231 4 11

5 db-engines.com/en/system/Neo4j%3BTtitan – Ashraful Islam Jul 27 '17 at 10:12

3 Answers

14

Titan was originally backed by Aurelius, which was bought by DataStax in 2015. This move was designed to give DataStax a jump-start into the Graph DB world, as they now offer their own "DSE Graph" enterprise product. Titan was since been forked (as previously mentioned) into JanusGraph.

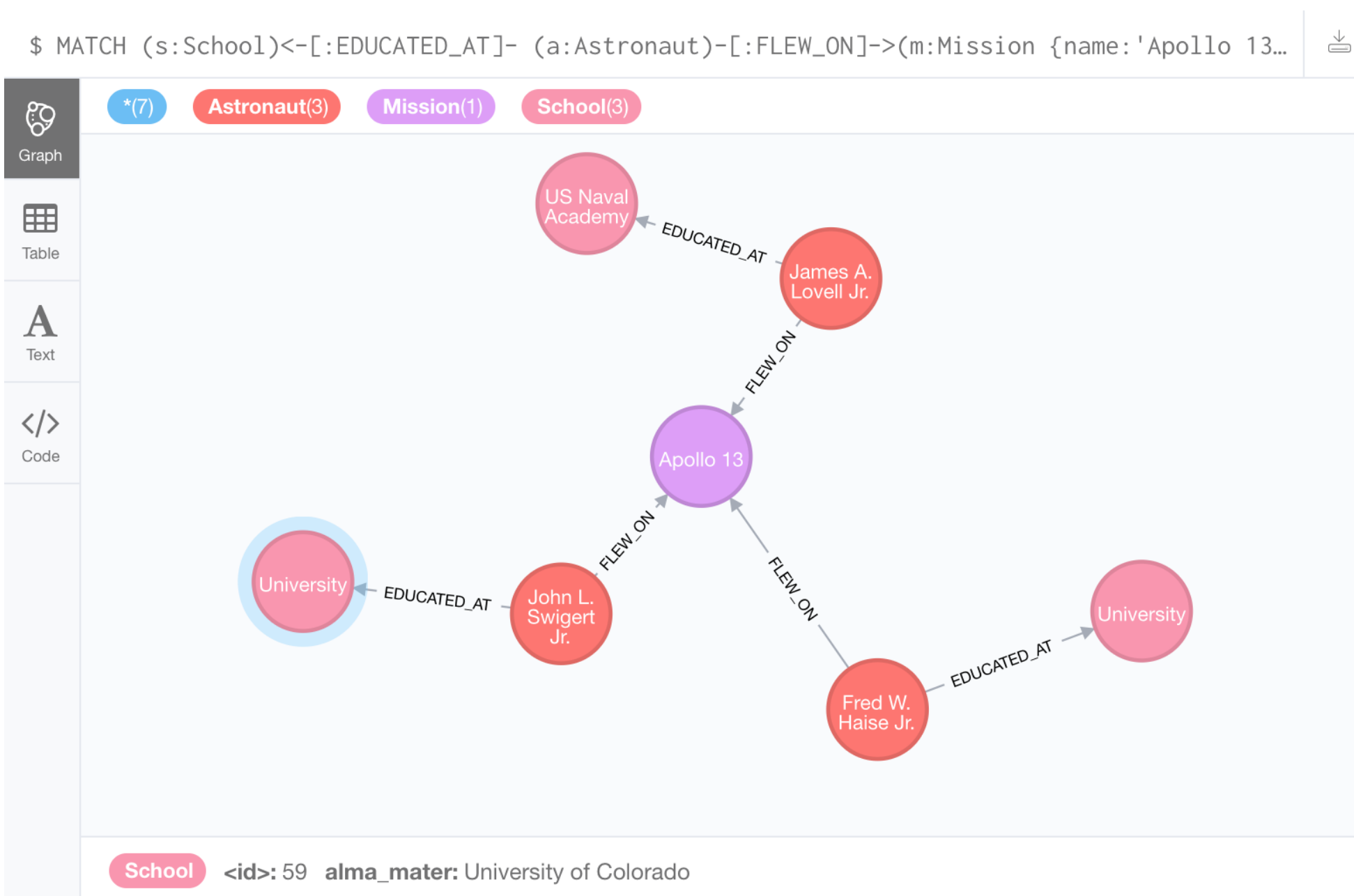
The nice thing about Titan/Janus (IMO) is that it is "pluggable" with other existing back-end and search technologies. So it will "play nice" with things like Cassandra, HBase, Hadoop, Solr, and ElasticSearch.

The drawback is that the community support is tough. The Titan project has been effectively killed, and Janus scores a whopping 0.23 on DBEngines. That makes it the 16th most-popular Graph DB (231st overall), which is pretty low.

Neo4j is backed by Neo Technology, and is regarded as the front-runner in the Graph DB community (score of 38.52 right now, 1st graph DB and 21st overall). It is open source, but controlled by Neo Technologies so they can dictate a difference in feature set between open source and enterprise.

The nice thing about Neo4j is that they have a lot of tutorials and learning aids built right-in to the Neo4j Browser, which is a nice, user-friendly web interface. Their documentation is top-notch, easy to read and search through, and they have a pretty good following here on Stack Overflow.

Neo4j Browser screenshot:



The drawback of Neo4j, is that some features (like clustering) are only available in the enterprise version. But if you work for a big company who doesn't mind shelling-out $ for an enterprise license, that may not be a big deal.

Consistency: Titan/Janus is a part of the "eventual consistency" crowd, while Neo4j aims to be strong-consistent (especially in a causal clustering scenario). Although consistency can be tuned with configuration in both, with Titan/Janus that can be dependent on your choice of pluggable backend (ex: typically strong-consistent with HBase, while eventually consistent with Cassandra).

Recommendation:

If you're just starting to learn graph databases and modeling, you can't go wrong with Neo4j. Simply download/install the community edition, run it, and execute `:play movies` as your first command (tutorial that walks you through loading, modeling, and querying movie relationships).

If you have some experience with graph, and you don't mind troubleshooting/googling to figure out things (like how to set the max frame size for Thrift), then you could probably do some really cool things with Titan.

Try each out, and see which one works for you.

edited Aug 11 '17 at 15:57

answered Jul 27 '17 at 15:39

Aaron
36.5k 10 81 104

15

One approach is to simply try to choose one database over the other. For example, you might quickly search around to find that Titan has been forked to JanusGraph where it is more actively maintained. In your research you may find that there are other open source graph databases as well like OrientDb, ChronoGraph, or Sqlg as well as commercial alternatives like Microsoft's CosmosDb, DSE Graph or IBM Graph. How do you decide now?

There is a graph framework that ties together all of these graphs including Neo4j/Titan (and more than those listed here): Apache TinkerPop. TinkerPop provides an abstraction over different graph databases and graph processors allowing the same code to be used with different configurable backends. This pattern is quite similar to the one you find in SQL with JDBC which helps make your code vendor agnostic.

You can try all of the different supported graph databases before you make a choice and you can do this type of prototyping/benchmarking fairly quickly with the Gremlin Console. You will be able to make self-informed choice as to what is the best way to go for your project.

It occurs to me as I come to the end of this post that I haven't directly answered your question. If you are just getting started and are just interested in learning about graph databases, then I likely wouldn't recommend starting with Titan/JanusGraph as it requires a bit of configuration to get started (schemas, backend selection, etc). Start with TinkerGraph or Neo4j using the Gremlin Console to try out some simple graph traversals and go from there.

edited Jul 27 '17 at 10:36

answered Jul 27 '17 at 10:28

stephen mallette
28.9k 4 35 86

Tinkerpop is not like the JDBC protocol, but more like an ORM (ex: Hibernate) that speak/implement Gremlin with most of graph databases (and every Graph database are not Gremlin centric). – logisima Jul 28 '17 at 0:57

The JDBC analogy is perfectly acceptable one to help convey the idea that TinkerPop helps promote a vendor agnostic way to develop graph applications. I was not implying that TinkerPop was a "standard" and by my simply mentioning JDBC as analogy I'm not sure how you would get to that conclusion. You'd have to explain your reasoning further that TinkerPop is like an ORM....I don't see the connection. – stephen mallette Aug 5 '17 at 11:53

9

There are far more than two graph databases - there are dozens. That being said, there are two with real market share: Neo4j and Titan/JanusGraph. But there are dozens of other graph datases, each with interesting strengths for different specific application spaces. That being said, I wouldn't dig into all of the niche players to start with - learning the basic idea of graph databases can be done with one of the two lead players.

Neo4j is the most mature, with the most nicely packaged install and documentation, tons of reference code, and support from a wide range of partners.

Titan/JanusGraph is the next most popular, as it's free/open source and has very strong support (e.g. IBM, Google, Hortonworks, AWS, ...). There's a recent complexity in that the leaders of the Titan project were acquired, freezing the Titan project. But the community forked the project into JanusGraph. So while JanusGraph is a new project, it's literally the same Titan code, with even broader industry support than Titan had.

Related to the two is the language used to work with the graphs. Neo4j uses its proprietary language, Cypher, while nearly everyone else uses Gremlin, and the TinkerPop open source tool set (which is a part of the Apache set of open source projects). Nearly all graph databases, including Neo4j, support Gremlin and TinkerPop. So, for example, you can use either Cypher or Gremlin to query Neo4j, though Neo (and some other proprietary graph database vendors) support Gremlin as a second-class citizen, so to speak. For example, you can connect to Neo using Gremlin from the (external) Gremlin console, but you can't use Gremlin in the (very nice) Neo4j console.

Note that there are many graph databases that support Gremlin other than Titan/JanusGraph. One new entrant that's very interesting is Microsoft's Azure Cosmos DB, which is a managed graph database that's "cheap and easy" if you use Azure already. And there are several vendors that provide managed JanusGraph.

For personal learningk I'd say that Neo4j is the easiest to set up and learn - you download and run it, and open a web browser onto their web-based console, which only takes a few minutes. That being said, if you're comfortable on a command line JanusGraph only took a half hour to install and get running for me, so it's not too hard.

For learning the concepts Neo4j is great. Neo4j's query language, Cypher, and JanusGraph's query language, Gremlin, are semantically identical, just spelled differently, so you'll learn the concepts either way.

For building a real system, either could work (and there are many successful following both approaches).

For which you choose, you'll want to think about whether you want to be strategically tied to a single vendor (Neo4j) or in a broader standards-based community. There's comfort level in picking the market leader with the most mature product - Neo4j. And there's a comfort level in picking open standards with strong industry support - JanusGraph. So IMO there's no "wrong" answer - people using either one are happy and successful. But since you have to pick, you'll need to think about which you're more comfortable with long-term.

answered Aug 18 '17 at 16:36

Laird Popkin
151 1 2